

LAPORAN IMPLEMENTASI DAN UJI COBA SIMULASI LOAD BALANCER NGINX MENGGUNAKAN DOCKER COMPOSE

Dosen Pengampuh Mata Kuliah : RUNAL REZKIAWAN S.Kom.,M.T



DISUSUN OLEH :

NAMA :

NIM :

KELAS :

PROGRAM STUDI INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS MUHAMMADIYAH MAKASSAR

2026

KATA PENGANTAR

Puji syukur penulis panjatkan kehadirat Tuhan Yang Maha Esa karena atas rahmat-Nya laporan implementasi sistem ini dapat diselesaikan dengan baik. Laporan ini disusun untuk memenuhi tugas mata kuliah Sistem Terdistribusi, dengan fokus utama pada perancangan, pengujian, dan analisis perbandingan algoritma *Load Balancing*—yaitu *Round Robin* dan *Least Connection*—yang diimplementasikan pada replika layanan akademik menggunakan teknologi kontainer Docker dan web server Nginx. Penulis berharap laporan ini dapat memberikan wawasan praktis mengenai manajemen trafik pada jaringan komputer dan sistem terdistribusi modern.

DAFTAR ISI

KATA PENGANTAR	i
DAFTAR ISI.....	ii
BAB I.....	1
PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	2
1.3 Tujuan	2
1.4 Manfaat	2
1.5 Batasan Masalah	3
BAB II.....	4
LANDASAN TEORI.....	4
2.1 Load Balancer	4
2.2 Algoritma Round Robin.....	4
2.3 Algoritma Least Connection	4
2.4 Docker dan NGINX	4
BAB III	6
PERANCANGAN DAN IMPLEMENTASI SISTEM.....	6
3.1 Arsitektur Sistem	6
3.2 Struktur Folder Program	6
3.3 Implementasi Docker Compose	7
3.4 Konfigurasi Round Robin.....	9
3.5 Konfigurasi Least Connection	10
3.6 Script Pengujian	11
BAB IV	12
HASIL DAN PEMBAHASAN.....	12
4.1 Skenario Pengujian	12

4.2 Hasil Pengujian Least Connection	12
4.3 Analisis Hasil Least Connection	13
4.4 Hasil Pengujian Round Robin.....	13
4.5 Analisis Hasil Round Robin.....	14
4.6 Perbandingan Least Connection dan Round Robin	14
4.7 Bukti Keberhasilan Pengujian.....	15
BAB V	20
PENUTUP	20
5.1 Kesimpulan	20
5.2 Saran	21
DAFTAR PUSTAKA	22

BAB I

PENDAHULUAN

1.1 Latar Belakang

Pada lingkungan institusi perguruan tinggi, sistem informasi akademik merupakan layanan vital yang diakses secara masif oleh ribuan mahasiswa secara simultan. Pada momen-momen tertentu, seperti periode Kontrak Rencana Studi (KRS), pengumuman nilai ujian, maupun pengaksesan jadwal kuliah, server akademik sering kali mengalami lonjakan trafik yang ekstrem. Jika infrastruktur hanya bergantung pada satu server tunggal, hal tersebut akan memicu *overload* yang mengakibatkan sistem menjadi *crash (single point of failure)*.

Untuk menjamin ketersediaan layanan yang tinggi (*high availability*), diperlukan implementasi *Load Balancing* untuk membagi beban trafik masuk ke beberapa replika server backend secara adil. Dalam perkembangan arsitektur modern, implementasi ini sangat efisien jika dibangun di atas teknologi kontainerisasi seperti Docker. Docker memudahkan *deployment* banyak server terisolasi dalam satu mesin fisik melalui Docker Compose.

Laporan ini membahas perancangan kluster sistem akademik tiruan berbasis Node.js yang terdiri dari Server KRS Online, Server Nilai Mahasiswa, dan Server Jadwal Kuliah. Sistem ini diintegrasikan dengan Nginx sebagai *Load Balancer* untuk menguji dan menganalisis efisiensi dua metode distribusi beban, yaitu *Round Robin* dan *Least Connection*.

1.2 Rumusan Masalah

- 1 Bagaimana cara merancang dan mengonfigurasi arsitektur *Load Balancer* Nginx yang terintegrasi dengan kluster mikro layanan akademik (KRS, Nilai, Jadwal) berbasis Node.js menggunakan Docker Compose?
- 2 Bagaimana karakteristik, pola respons, dan perbedaan mekanis dari algoritma *Round Robin* dan *Least Connection* saat menerima permintaan (*request*) sekuensial dari skrip pengujian?

1.3 Tujuan

- 1 Membangun lingkungan *multi-container* yang mengisolasi aplikasi web akademik Node.js ke dalam tiga backend server serta mengonfigurasi proksi Nginx.
- 2 Menganalisis dan membandingkan akurasi pembagian beban trafik antara metode *Round Robin* dan *Least Connection* berdasarkan log output sistem.

1.4 Manfaat

- 1 Memberikan visualisasi nyata mengenai teknik *scaling* horizontal pada sistem terdistribusi skala akademik.
- 2 Memberikan referensi bagi administrator jaringan dalam menentukan algoritma penyeimbangan beban terbaik saat menghadapi pola trafik layanan yang bervariasi.

1.5 Batasan Masalah

1. Implementasi dan pengujian dijalankan pada lingkungan lokal (*localhost*) menggunakan Docker Desktop dan ekstensi Docker Compose pada Visual Studio Code.
2. Aplikasi backend dibangun menggunakan komponen Node.js yang dikemas melalui berkas `Dockerfile`.
3. Parameter pengujian dibatasi pada evaluasi respons sekuensial HTTP Layer 7 menggunakan skrip otomatisasi PowerShell.

BAB II

LANDASAN TEORI

2.1 Load Balancer

Load Balancer adalah komponen arsitektur jaringan yang bertindak sebagai proksi terbalik (*reverse proxy*) di depan sekumpulan server (kluster). Tugas utamanya adalah menerima seluruh permintaan masuk dari klien, mengevaluasi aturan distribusi berdasarkan algoritma tertentu, dan meneruskan trafik tersebut ke server backend yang paling optimal. Hal ini dilakukan untuk mengoptimalkan utilitas sumber daya dan mempercepat *response time*.

2.2 Algoritma Round Robin

Round Robin adalah algoritma penyeimbangan beban statis yang mendistribusikan trafik masuk ke setiap server tujuan secara berurutan sesuai daftar antrian tanpa memperhitungkan variasi kapasitas komputasi maupun beban kerja dinamis pada server backend. Kelebihannya terletak pada kesederhanaan proses komputasi yang sangat ringan pada sisi *Load Balancer*.

2.3 Algoritma Least Connection

Least Connection adalah algoritma penyeimbangan beban dinamis yang bekerja dengan cara memantau tabel status koneksi aktif pada setiap server secara *real-time*. Permintaan baru dari klien akan selalu diarahkan ke server backend yang saat itu memiliki jumlah koneksi aktif paling sedikit. Metode ini sangat ideal untuk jenis request yang membutuhkan waktu pemrosesan berbeda (heterogen).

2.4 Docker dan NGINX

- **Docker & Docker Compose:** Docker adalah platform kontainerisasi untuk membungkus kode aplikasi, *runtime*, dan *library* ke dalam satu *image* independen.

Docker Compose menyederhanakan pengelolaan arsitektur banyak kontainer melalui satu berkas skrip berekstensi `.yaml`.

- **NGINX:** Perangkat lunak web server yang tangguh dengan arsitektur *event-driven*. Nginx sangat andal digunakan sebagai *Load Balancer* Layer 7 (HTTP) karena kemampuannya memproses puluhan ribu trafik serentak dengan konsumsi memori yang sangat efisien.

BAB III

PERANCANGAN DAN IMPLEMENTASI SISTEM

3.1 Arsitektur Sistem

Sistem ini dirancang sebagai purwarupa kluster server akademik. Trafik dari klien diarahkan menuju kontainer Nginx yang memproksikan permintaan ke tiga kontainer aplikasi Node.js internal di dalam satu jaringan virtual Docker. Setiap kontainer backend mewakili sub-layanan spesifik:

- app1 Server KRS Online
- app2 Server Nilai Mahasiswa
- app3 Server Jadwal Kuliah

3.2 Struktur Folder Program

Sesuai dengan implementasi pada Visual Studio Code, struktur direktori proyek diatur sebagai berikut:

KLP2-LOAD-BALANCER-AKADEMIK/

```
|— app/
|   |— Dockerfile
|   |— package.json
|   |— server.js
|— nginx/
|   |— nginx-least-connection.conf
```

```
| └─ nginx.conf
|
└─ scripts/
    | └─ test-least-connection.ps1
    | └─ test-round-robin.ps1
    └─ docker-compose.yml
```

3.3 Implementasi Docker Compose

Spesifikasi interkoneksi seluruh layanan didefinisikan pada file `docker-compose.yml`:

```
services:

  app1:
    build: ./app
    container_name: app1
    environment:
      - SERVER_NAME=Server KRS Online
      - SERVER_ID=app1
      - PORT=3000
      - LOAD_BALANCER_ALGORITHM=Least Connection
      - LOAD_BALANCER_LAYER=Layer 7

  app2:
    build: ./app
    container_name: app2
```

```

environment:
  - SERVER_NAME=Server Nilai Mahasiswa
  - SERVER_ID=app2
  - PORT=3000
  - LOAD_BALANCER_ALGORITHM=Least Connection
  - LOAD_BALANCER_LAYER=Layer 7

app3:
  build: ./app
  container_name: app3
  environment:
    - SERVER_NAME=Server Jadwal Kuliah
    - SERVER_ID=app3
    - PORT=3000
    - LOAD_BALANCER_ALGORITHM=Least Connection
    - LOAD_BALANCER_LAYER=Layer 7

nginx:
  image: nginx:latest
  container_name: nginx-lb

  ports:
    - "80:80"

  volumes:
    - ./nginx/nginx-least-connection.conf:/etc/nginx/nginx.conf

```

```
depends_on:
```

- app1
- app2
- app3

3.4 Konfigurasi Round Robin

Isi dari file konfigurasi `nginx/nginx.conf`:

```
events {}

http {

    upstream akademik_backend {

        server app1:3000;
        server app2:3000;
        server app3:3000;

    }

    server {

        listen 80;

        location / {

            proxy_pass http://akademik_backend;
```

```

    }
}
}

```

3.5 Konfigurasi Least Connection

Isi kode konfigurasi di dalam file `nginx-least-connection.conf`:

```

Write-Host ""
Write-Host "=====
Write-Host "UJI LOAD BALANCER : LEAST CONNECTION"
Write-Host "=====

for ($i=1; $i -le 10; $i++) {

    $hasil = Invoke-RestMethod "http://localhost/?delay=3000"

    Write-Host ""
    Write-Host "Request ke-$i"

    Write-Host "Algorithm      : Least Connection"
    Write-Host "Layer              : Layer 7"
    Write-Host "Backend            : $($hasil.server_id)"
    Write-Host "Server Name       : $($hasil.server_name)"
    Write-Host "Status            : $($hasil.status)"
}

```

3.6 Script Pengujian

Pengujian otomatis dilakukan memanfaatkan skrip PowerShell (.ps1) pada folder scripts/ untuk mengirimkan deretan iterasi request HTTP GET secara konstan menggunakan perintah Invoke-WebRequest di terminal VS Code.

BAB IV

HASIL DAN PEMBAHASAN

4.1 Skenario Pengujian

Pengujian dilakukan dengan memastikan kluster kontainer berjalan melalui eksekusi perintah docker compose up -d. Skrip pengujian test-round-robin.ps1 dan test-least-connection.ps1 dijalankan berturut-turut untuk merekam data respons dari Layer 7, nama Backend, nama Layanan Akademik (*Server Name*), dan status jaringan.

4.2 Hasil Pengujian Least Connection

Eksekusi file skrip .\scripts\test-least-connection.ps1 menghasilkan log terminal

```
PS D:\SEMESTER 6\k1p2-load-balancer-akademik> .\scripts\test-least-connection.ps1
=====
UI: LOAD BALANCER : LEAST CONNECTION
=====

Request ke-1
Algorithm : Least Connection
Layer     : Layer 7
Backend   : app1
Server Name : Server KRS Online
Status    : ACTIVE

Request ke-2
Algorithm : Least Connection
Layer     : Layer 7
Backend   : app2
Server Name : Server Nilai Mahasiswa
Status    : ACTIVE

Request ke-3
Algorithm : Least Connection
Layer     : Layer 7
Backend   : app3
Server Name : Server Jadwal Kuliah
Status    : ACTIVE

Request ke-4
Algorithm : Least Connection
Layer     : Layer 7
Backend   : app1
Server Name : Server KRS Online
Status    : ACTIVE

Request ke-5
Algorithm : Least Connection
Layer     : Layer 7
Backend   : app2
Server Name : Server Nilai Mahasiswa
Status    : ACTIVE

Request ke-6
Algorithm : Least Connection
Layer     : Layer 7
Backend   : app3
Server Name : Server Jadwal Kuliah
Status    : ACTIVE

Request ke-7
Algorithm : Least Connection
Layer     : Layer 7
Backend   : app1
Server Name : Server KRS Online
Status    : ACTIVE

Request ke-8
Algorithm : Least Connection
Layer     : Layer 7
Backend   : app2
Server Name : Server Nilai Mahasiswa
Status    : ACTIVE

Request ke-9
Algorithm : Least Connection
Layer     : Layer 7
Backend   : app3
Server Name : Server Jadwal Kuliah
Status    : ACTIVE

Request ke-10
Algorithm : Least Connection
```


4.3 Analisis Hasil Least Connection

Berdasarkan pembacaan log pengujian dinamis, penyeimbangan beban berjalan sangat optimal. Permintaan dialihkan secara cerdas ke kontainer yang memiliki beban koneksi aktif paling minim. Karena pengujian dilakukan secara berurutan dengan interval eksekusi yang konsisten, beban koneksi di setiap kontainer backend Node.js langsung turun kembali ke angka nol setelah membalas data, sehingga distribusi terlihat terbagi rata ke seluruh server backend.

4.4 Hasil Pengujian Round Robin

Eksekusi file skrip `.\scripts\test-round-robin.ps1` menghasilkan urutan log terminal

```
PS D:\SEMESTER 6\ktp-load-balancer-akademik> .\scripts\test-round-robin.ps1
=====
UJI LOAD BALANCER : ROUND ROBIN
=====

Request ke-1
Algorithm   : Round Robin
Layer       : Layer 7
Backend     : app1
Server Name : Server KRS Online
Status      : ACTIVE

Request ke-2
Algorithm   : Round Robin
Layer       : Layer 7
Backend     : app2
Server Name : Server Nilai Mahasiswa
Status      : ACTIVE

Request ke-3
Algorithm   : Round Robin
Layer       : Layer 7
Backend     : app3
Server Name : Server Jadwal Kuliah
Status      : ACTIVE

Request ke-4
Algorithm   : Round Robin
Layer       : Layer 7
Backend     : app1
Server Name : Server KRS Online
Status      : ACTIVE

Request ke-5
Algorithm   : Round Robin
Layer       : Layer 7
Backend     : app2
Server Name : Server Nilai Mahasiswa
Status      : ACTIVE

Request ke-6
Algorithm   : Round Robin
Layer       : Layer 7
Backend     : app3
Server Name : Server Jadwal Kuliah
Status      : ACTIVE

Request ke-7
Algorithm   : Round Robin
Layer       : Layer 7
Backend     : app1
Server Name : Server KRS Online
Status      : ACTIVE

Request ke-8
Algorithm   : Round Robin
Layer       : Layer 7
Backend     : app2
Server Name : Server Nilai Mahasiswa
Status      : ACTIVE

Request ke-9
Algorithm   : Round Robin
Layer       : Layer 7
Backend     : app3
Server Name : Server Jadwal Kuliah
Status      : ACTIVE

Request ke-10
Algorithm   : Round Robin
```

4.5 Analisis Hasil Round Robin

Log tersebut membuktikan bekerjanya mekanisme sirkular kaku khas dari algoritma *Round Robin*. Permintaan diteruskan secara adil dan bergiliran tanpa celah dari app3 menuju app1, berlanjut ke app2, lalu kembali lagi ke app3 secara siklis. Algoritma ini sangat andal menjamin distribusi trafik yang seimbang mutlak asalkan beban kerja dari setiap fitur akademik tersebut seragam.

4.6 Perbandingan Least Connection dan Round Robin

Berikut adalah tabel perbandingan karakteristik antara algoritma *Round Robin* dan *Least Connection* berdasarkan hasil analisis dan uji coba sistem:

Parameter Evaluasi	Algoritma Round Robin	Algoritma Least Connection
Sifat Distribusi	Statis (Berdasar urutan daftar antrean).	Dinamis (Berdasar beban koneksi aktif riil).
Kriteria Pengalihan	Murni bergiliran secara sirkular tanpa melihat kondisi server.	Diarahkan ke server dengan jumlah koneksi aktif paling sedikit.
Efisiensi Sumber Daya	Kurang efektif jika ada salah satu server yang mengalami <i>stuck</i> proses panjang.	Sangat efektif mencegah penumpukan trafik pada satu server yang sedang sibuk.
Kompleksitas	Sangat rendah karena	Sedikit lebih tinggi karena

Parameter Evaluasi	Algoritma Round Robin	Algoritma Least Connection
Proses	tidak memerlukan <i>monitoring</i> metrik.	Nginx harus memantau status koneksi secara <i>real-time</i> .
Perilaku pada Trafik Rendah	Tetap membagi beban secara berurutan dan merata.	Menghasilkan pola sebaran yang mirip dengan Round Robin karena koneksi aktif cepat kembali ke nol.
Rekomendasi Kasus Penggunaan	Sangat cocok untuk layanan web statis dengan beban kerja server yang homogen.	Sangat cocok untuk sistem akademik dinamis dengan beban kerja bervariasi (misal: transaksi database berat).

4.7 Bukti Keberhasilan Pengujian

Tahap 1: Inisialisasi dan Menyalakan Kluster Kontainer (Round Robin)

```

PS D:\SEMESTER 6\klp2-load-balancer-akademik> docker compose up --build -d
[+] up 10/10
✓ Image nginx:latest Pulled
#1 [internal] load local bake definitions
#1 reading from stdin 1.57kB 0.1s done
#1 DONE 0.1s

#2 [app3 internal] load build definition from Dockerfile
#2 transferring dockerfile:
#2 transferring dockerfile: 174B 0.0s done
#2 DONE 0.2s

#3 [app2 internal] load metadata for docker.io/library/node:18-alpine
#3 DONE 3.5s

#4 [app1 internal] load .dockerignore
#4 transferring context: 2B 0.0s done
#4 DONE 0.1s

#4 [app2 internal] load .dockerignore
#4 DONE 0.1s

#5 [app1 internal] load build context
#5 transferring context: 1.07kB 0.1s done
#5 DONE 0.4s

#6 [app3 1/5] FROM docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc
#6 resolve docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f2
#6 DONE 0.5s

```

Tahap 2: Verifikasi Status Keaktifan Kontainer

```

PS D:\SEMESTER 6\klp2-load-balancer-akademik> docker ps

```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAME
5af2eb648d2e	nginx:latest	"/docker-entrypoint..."	23 seconds ago	Up 22 seconds	0.0.0.0:80->80/tcp, [::]:80->80/tcp	nginx
5fb43849e36b	klp2-load-balancer-akademik-app3	"docker-entrypoint.s..."	24 seconds ago	Up 22 seconds	3000/tcp	app3
7873f97aa417	klp2-load-balancer-akademik-app1	"docker-entrypoint.s..."	24 seconds ago	Up 22 seconds	3000/tcp	app1
68775b713f43	klp2-load-balancer-akademik-app2	"docker-entrypoint.s..."	24 seconds ago	Up 22 seconds	3000/tcp	app2

```

PS D:\SEMESTER 6\klp2-load-balancer-akademik> .\scripts\test-least-connection.ps1

```

Tahap 3: Eksekusi Skrip Pengujian Algoritma Round Robin

```

PS D:\SEMESTER 6\klp2-load-balancer-akademik> .\scripts\test-round-robin.ps1

=====
UJI LOAD BALANCER : ROUND ROBIN
=====

Request ke-1
Algorithm   : Round Robin
Layer       : Layer 7
Backend     : app1
Server Name : Server KRS Online
Status      : ACTIVE

Request ke-2
Algorithm   : Round Robin
Layer       : Layer 7
Backend     : app2
Server Name : Server Nilai Mahasiswa
Status      : ACTIVE

Request ke-3
Algorithm   : Round Robin
Layer       : Layer 7
Backend     : app3
Server Name : Server Jadwal Kuliah
Status      : ACTIVE

Request ke-4
Algorithm   : Round Robin
Layer       : Layer 7
Backend     : app1
Server Name : Server KRS Online
Status      : ACTIVE

Request ke-5
Algorithm   : Round Robin
Layer       : Layer 7
Backend     : app2
Server Name : Server Nilai Mahasiswa
Status      : ACTIVE

```

Tahap 4: Menghentikan Sistem Sementara

```

PS D:\SEMESTER 6\klp2-load-balancer-akademik> docker compose down
[+] down 5/5
✓ Container nginx-lb          Removed
  0.6s
✓ Container app3              Removed
  1.6s
✓ Container app1              Removed
  1.8s
✓ Container app2              Removed
  1.8s
✓ Network klp2-load-balancer-akademik_default Removed
  0.3s

```

Tahap 6: Menyalakan Ulang Kluster Kontainer (Least Connection)

```

PS D:\SEMESTER 6\klp2-load-balancer-akademik> docker compose up --build -d
#1 [internal] load local bake definitions
#1 reading from stdin 1.57kB 0.1s done
#1 DONE 0.1s

#2 [app1 internal] load build definition from Dockerfile
#2 transferring dockerfile: 174B 0.0s done
#2 DONE 0.1s

#3 [app1 internal] load metadata for docker.io/library/node:18-alpine
#3 DONE 2.8s

#4 [app1 internal] load .dockerignore
#4 transferring context: 2B 0.0s done
#4 DONE 0.1s

#5 [app1 internal] load build context
#5 transferring context: 93B 0.0s done
#5 DONE 0.1s

```

Tahap 7: Eksekusi Skrip Pengujian Algoritma Least Connection

```

PS D:\SEMESTER 6\klp2-load-balancer-akademik> .\scripts\test-least-connection.ps1

=====
UJI LOAD BALANCER : LEAST CONNECTION
=====

Request ke-1
Algorithm   : Least Connection
Layer       : Layer 7
Backend     : app1
Server Name : Server KRS Online
Status      : ACTIVE

Request ke-2
Algorithm   : Least Connection
Layer       : Layer 7
Backend     : app2
Server Name : Server Nilai Mahasiswa
Status      : ACTIVE

Request ke-3
Algorithm   : Least Connection
Layer       : Layer 7
Backend     : app3
Server Name : Server Jadwal Kuliah
Status      : ACTIVE

Request ke-4
Algorithm   : Least Connection
Layer       : Layer 7
Backend     : app1
Server Name : Server KRS Online
Status      : ACTIVE

Request ke-5
Algorithm   : Least Connection
Layer       : Layer 7
Backend     : app2
Server Name : Server Nilai Mahasiswa
Status      : ACTIVE

```

Tahap 8: Penghentian Akhir Seluruh Sistem

```
● PS D:\SEMESTER 6\klp2-load-balancer-akademik> docker compose down
[+] down 5/5
✓ Container nginx-lb          Removed
✓ Container app3              Removed
✓ Container app1              Removed
✓ Container app2              Removed
✓ Network klp2-load-balancer-akademik_default Removed
○ PS D:\SEMESTER 6\klp2-load-balancer-akademik>
```

BAB V

PENUTUP

5.1 Kesimpulan

Berdasarkan hasil perancangan dan implementasi yang telah dilakukan, dapat disimpulkan bahwa sistem *load balancing* menggunakan Docker dan NGINX berhasil dibuat dan dijalankan dengan baik. Sistem ini terdiri dari tiga backend server yang berjalan dalam container Docker, yaitu Server KRS Online, Server Nilai Mahasiswa, dan Server Jadwal Kuliah, yang dihubungkan melalui NGINX sebagai *load balancer*.

Implementasi menunjukkan bahwa NGINX mampu mendistribusikan *request* ke beberapa server menggunakan dua algoritma, yaitu *Round Robin* dan *Least Connection*. Pada metode *Round Robin*, distribusi *request* dilakukan secara bergiliran ke setiap server secara merata tanpa mempertimbangkan beban server. Sedangkan pada metode *Least Connection*, *request* diarahkan ke server dengan jumlah koneksi aktif paling sedikit sehingga lebih optimal dalam kondisi beban tinggi.

Hasil pengujian membuktikan bahwa kedua algoritma dapat berjalan dengan baik sesuai konfigurasi yang diterapkan pada NGINX di dalam container Docker. Docker berhasil mempermudah proses *deployment* dengan mensimulasikan beberapa server dalam satu lingkungan lokal tanpa membutuhkan banyak perangkat fisik. Penggunaan Docker dan NGINX terbukti efektif dalam mengimplementasikan sistem *load balancing* sederhana, serta memberikan pemahaman yang jelas mengenai perbedaan mekanisme kerja antara *Round Robin* dan *Least Connection* dalam distribusi beban server.

5.2 Saran

Untuk pengembangan sistem ke depan, disarankan melakukan simulasi pengujian dengan beban trafik konkuren tingkat tinggi menggunakan alat uji beban (*stress-testing tool*) seperti Apache JMeter. Hal ini diperlukan untuk melihat performa batas maksimal serta perilaku nyata pengalihan trafik dari algoritma *Least Connection* secara lebih mendalam ketika salah satu server sengaja diberi beban kerja komputasi tiruan yang berat.

DAFTAR PUSTAKA

Docker Documentation. (2025). *Docker Compose Specification and Multi-Container Services*. Docker Inc.

Nginx Technical Reference. (2024). *HTTP Load Balancing Architecture and Directives*. Nginx Core Team.

Stallings, W. (2019). *Data and Computer Communications (10th Edition)*. Pearson.